

Homework Exercise Sheet

Ex. 1 Solution

Label (S) : only one correct answer; (M) : more than one correct answer

1. (S) Given $\mathbf{A} \in \mathbb{R}^{4 \times 4}$, which is the correct operation to compute $\mathbf{B} \in \mathbb{R}^{4 \times 4}$ through **adding the first row of $\mathbf{A}$ to its third row?**

$$\text{a) } \mathbf{B} = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \mathbf{A}$$

$$\text{b) } \mathbf{B} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \mathbf{A}$$

$$\text{c) } \mathbf{B} = \mathbf{A} \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$\text{d) } \mathbf{B} = \mathbf{A} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Solution:

b). It can be easily verified by direct computation:

$$\mathbf{B} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} A_{11} & A_{12} & A_{13} & A_{14} \\ A_{21} & A_{22} & A_{23} & A_{24} \\ A_{31} & A_{12} & A_{33} & A_{34} \\ A_{41} & A_{12} & A_{43} & A_{44} \end{bmatrix} = \begin{bmatrix} A_{11} & A_{12} & A_{13} & A_{14} \\ A_{21} & A_{22} & A_{23} & A_{24} \\ A_{31} + A_{11} & A_{32} + A_{12} & A_{33} + A_{13} & A_{34} + A_{14} \\ A_{41} & A_{42} & A_{43} & A_{44} \end{bmatrix}$$

A better solution would be interpreting $\mathbf{A}$ in row-wise and applying the usual matrix-vector multiplication:

$$\mathbf{B} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \mathbf{A}_{1,:} \\ \mathbf{A}_{2,:} \\ \mathbf{A}_{3,:} \\ \mathbf{A}_{4,:} \end{bmatrix} = \begin{bmatrix} \mathbf{A}_{1,:} \\ \mathbf{A}_{2,:} \\ \mathbf{A}_{3,:} + \mathbf{A}_{1,:} \\ \mathbf{A}_{4,:} \end{bmatrix}$$

2. (S) The Strassen algorithm is an algorithm for matrix multiplication of $(n \times n)$ -size matrices. It enjoys a better asymptotic complexity $\mathcal{O}(n^{\log_2 7})$ (for multiplication) instead of $\mathcal{O}(n^3)$ in the case of “straightforward” matrix multiplication. Given two matrices $\mathbf{A}, \mathbf{B} \in \mathbb{R}^{2l \times 2l}$, the Strassen algorithm partitions $\mathbf{A}, \mathbf{B}$ into equally $(l \times l)$ -size blocks:

$$\mathbf{A} = \begin{bmatrix} \mathbf{A}_{11} & \mathbf{A}_{12} \\ \mathbf{A}_{21} & \mathbf{A}_{22} \end{bmatrix}, \quad \mathbf{B} = \begin{bmatrix} \mathbf{B}_{11} & \mathbf{B}_{12} \\ \mathbf{B}_{21} & \mathbf{B}_{22} \end{bmatrix},$$

with $\mathbf{A}_{ij}, \mathbf{B}_{ij}, \mathbf{C}_{ij} \in \mathbb{R}^{l \times l}$, and defines the following submatrices:

$$\begin{aligned}\mathbf{M}_1 &= (\mathbf{A}_{11} + \mathbf{A}_{22})(\mathbf{B}_{11} + \mathbf{B}_{22}); \\ \mathbf{M}_2 &= (\mathbf{A}_{21} + \mathbf{A}_{22})\mathbf{B}_{11}; \\ \mathbf{M}_3 &= \mathbf{A}_{11}(\mathbf{B}_{12} - \mathbf{B}_{22}); \\ \mathbf{M}_4 &= \mathbf{A}_{22}(\mathbf{B}_{21} - \mathbf{B}_{11}); \\ \mathbf{M}_5 &= (\mathbf{A}_{11} + \mathbf{A}_{12})\mathbf{B}_{22}; \\ \mathbf{M}_6 &= (\mathbf{A}_{21} - \mathbf{A}_{11})(\mathbf{B}_{11} + \mathbf{B}_{12}); \\ \mathbf{M}_7 &= (\mathbf{A}_{12} - \mathbf{A}_{22})(\mathbf{B}_{21} + \mathbf{B}_{22}).\end{aligned}$$

Then resulting matrix $\mathbf{C} = \mathbf{AB}$ can be computed as:

$$\mathbf{C} = \begin{bmatrix} \mathbf{M}_1 + \mathbf{M}_4 - \mathbf{M}_5 + \mathbf{M}_7 & \mathbf{M}_3 + \mathbf{M}_5 \\ \mathbf{M}_2 + \mathbf{M}_4 & \mathbf{M}_1 - \mathbf{M}_2 + \mathbf{M}_3 + \mathbf{M}_6 \end{bmatrix}.$$

Given two matrices $\mathbf{A}, \mathbf{B} \in \mathbb{R}^{8 \times 8}$, how many element-wise multiplications are needed when computing $\mathbf{AB}$ using the Strassen algorithm?

Hint: For 2×2 matrices, Strassen's algorithm costs 7 multiplications.

- a) 512 b) 343 c) 236 d) 110

Solution:

b). By viewing $\mathbf{A}, \mathbf{B}$ as a 2×2 blocked matrix, this leads to 7 matrix-matrix multiplications of shape 4×4 . For each of the 4×4 matrix-matrix multiplications, we apply Strassen's algorithm again which needs 7 matrix-matrix multiplications of shape 2×2 . That in turn needs 7 element-wise matrix multiplications. In summary, a total of $343 (= 7^3)$ element-wise multiplications are needed.

3. (M) Consider the Gram-Schmidt process that takes a finite set of vectors $\{\mathbf{a}^1, \dots, \mathbf{a}^k\} \subset \mathbb{R}^n$ and generates an orthonormal set $\{\mathbf{q}^1, \dots, \mathbf{q}^l\} \subset \mathbb{R}^n$ such that $\text{span}\{\mathbf{a}^1, \dots, \mathbf{a}^l\} = \text{span}\{\mathbf{q}^1, \dots, \mathbf{q}^l\}$. Notice it might happen that $l < k$ since $\{\mathbf{a}^1, \dots, \mathbf{a}^k\}$ might be (almost) linearly dependent. The algorithm can be described in the following pseudo-code:

```

1:  $\mathbf{q}^1 := \frac{\mathbf{a}^1}{\|\mathbf{a}^1\|_2}$  % 1st output vector
2: for  $j = 2, \dots, k$  do
    { % Orthogonal projection
3:    $\mathbf{q}^j := \mathbf{a}^j$ 
4:   for  $\ell = 1, 2, \dots, j-1$  do
5:     {  $\mathbf{q}^j \leftarrow \mathbf{q}^j - \mathbf{a}^j \cdot \mathbf{q}^\ell \mathbf{q}^\ell$  }
6:     if ( $\mathbf{q}^j = \mathbf{0}$ ) then STOP
7:     else {  $\mathbf{q}^j \leftarrow \frac{\mathbf{q}^j}{\|\mathbf{q}^j\|_2}$  }
8:   }
```

A flawed implementation of the Gram-Schmidt orthonormalization is given below:

```

1 Eigen::Matrix gram_schmidt(const Eigen::MatrixXd &A) {
2     Eigen::MatrixXd Q(A);
3     Q.col(0).normalize();
4     for(unsigned int j = 1; j < A.cols(); ++j) {
5         Q.col(j) -= A.leftCols(j) * (Q.leftCols(j).transpose() * A.col(j));
6         if (Q.col(j).norm() == 0.0) {
7             break;
8         }
9         else {
10            Q.col(j).normalize();
11        }
12    }
13    return Q;
14 }
15

```

Which lines are wrong?

- a) line 4 b) line 5 c) line 6 d) line 10

Solution:

b), c). The correct implementation is:

line 5 → `Q.col(j) -= Q.leftCols(j) * (Q.leftCols(j).transpose() * A.col(j));`
line 6 → `if (Q.col(j).norm() <= 1e-9 * A.col(j).norm())`

4. **(S)** Given column vectors $\mathbf{a}, \mathbf{b} \in \mathbb{R}^n$, and $\mathbf{T}, \mathbf{M} \in \mathbb{R}^{n \times n}$, where $\mathbf{T}$ is a lower triangular matrix defined as

$$\mathbf{T} = \begin{bmatrix} 1 & 0 & 0 & \cdots & 0 \\ 1 & 1 & 0 & \cdots & 0 \\ \vdots & \ddots & \ddots & & \vdots \\ \vdots & & \ddots & \ddots & 0 \\ 1 & 1 & \cdots & 1 & 1 \end{bmatrix}.$$

What is the optimal complexity of computing $\mathbf{a}\mathbf{b}^T\mathbf{T}\mathbf{M}$?

Hint: The structure of $\mathbf{T}$ should be leveraged.

- a) $\mathcal{O}(n^3)$ b) $\mathcal{O}(n^2)$ c) $\mathcal{O}(n)$

Solution:

b). This question involves the tricks of improving complexity discussed in Sec. 1.4.3. The computation can be arranged in the order $\mathbf{a}(\mathbf{b}^T(\mathbf{T}\mathbf{M}))$. In the first step, the structure of $\mathbf{T}$ can be exploited. Each column vector of $\mathbf{T}\mathbf{M}$ is a vector of partial sums of components of each column vector of $\mathbf{M}$, which enjoys complexity $\mathcal{O}(n)$. Therefore, the computational effort of computing $\mathbf{T}\mathbf{M}$ is $\mathcal{O}(n^2)$. Both of the rest two steps enjoy complexity of $\mathcal{O}(n^2)$ obviously.

5. **(M)** Which of the following statements regarding floating point and machine precision are **wrong**?

- a) The spacing on the side of larger magnitude is smaller than the spacing on the side of smaller magnitude.
- b) Machine precision is the lower bound on the relative approximation error due to rounding in floating point arithmetic.
- c) Machine precision is the difference between 1 and the next larger floating point number.
- d) Machine precision for `double` is $\mathcal{O}(10^{-16})$.
- e) `(1.0 + 0.1 * std::numeric_limits<double>::epsilon() > 1.0)` is `True`.

Solution:

a), b), e). The right statements are:

- a) The spacing on the side of larger magnitude is **larger** than the spacing on the side of smaller magnitude.
- b) Machine precision is the **upper** bound on the relative approximation error due to rounding in floating point arithmetic.
- e) The expression is **False**.

6. (S) Consider a customized unsigned floating number format with

- basis $B = 10$
- exponent range $e_{\min} = -3, e_{\max} = 3$
- mantissa digit $m = 3$.

What are the largest and the smallest number (in modulus) in this form?

- a) 999,0.0001 b) 1000,0.0001 c) 999,0.001 d) 1000,0.001

Solution:

a). According to the definition of floating point numbers (Def. 1.5.2.4), the largest machine number is computed by $(1 - B^{-m}) \cdot B^{e_{\max}} = (1 - 0.001) \times 10^3 = 999$ and the smallest by $B^{-1} \cdot B^{e_{\min}} = 0.1 \times 10^{-3} = 0.0001$.

7. (S) To numerically compute the two roots (x_1, x_2) of quadratic polynomial $x \mapsto x^2 + \alpha x + \gamma$, which of the following the expressions avoid cancellation?

- a) $x_1 = \frac{-\alpha + \sqrt{\alpha^2 - 4\beta}}{2}, x_2 = -\alpha - x_1$
- b) $x_1 = \frac{-\alpha + \sqrt{\alpha^2 - 4\beta}}{2}, x_2 = \frac{\beta}{x_1}$
- c) Both of them avoid cancellation.
- d) None of them avoids cancellation.

Solution:

d). If $\alpha > 0$, the computation $x_1 = (-\alpha + \sqrt{\alpha^2 - 4\beta})/2$ could already incur cancellation. The right approach is to first check the sign of α , compute one of the roots that avoids cancellation, and then yield the second root via Vieta's formula. See the code 1.5.4.18.

8. **(S)** To numerically compute $\operatorname{arcsinh}(x) := \log(x + \sqrt{x^2 + 1})$, which of the following the expressions avoid cancellation?

- a) $\log(\sqrt{1+x^2} + x)$
- b) $-\log(\sqrt{1+x^2} - x)$
- c) Both of them avoids cancellation.
- d) None of them avoids cancellation.

Solution:

d). The right approach is to first check the sign of x and apply the functions in a) or b) accordingly.

9. **(S)** Which of the following finite difference formulas gives the most accurate approximation of the derivative of $f(x)$ at $x = 0.5$ where $f(x) = \sin(x)$? We assume $\delta x = 0.01$ in the following formulas.

- a) $\frac{f(x+\delta x) - f(x)}{\delta x}$
- b) $\frac{f(x) - f(x-\delta x)}{\delta x}$
- c) $\operatorname{Im}\left(\frac{f(x+i\delta x)}{\delta x}\right)$

Solution:

c). By calculating the values of each formula and compare them with the symbolic derivative $f'(x) = \cos(x)$, one can check that *c*) is more accurate than the forward difference in *a*) and backward difference in *b*) (see the discussion of **complex step approximation** in [this blog](#)). In fact, *c*) enjoys 2nd-order accuracy while the other two are only 1st-order accurate.